

konspekt / local

read-only view over a live project

SHIPPED · `implementation_zero`

konspekt now has a local UI

A window that sits beside the session and shows the project state as the agent and I write to it. Three tabs and a detail panel, updating live.

Changes

Stats

Goals

+ detail panel

A rolling to-do over every entity

A ten-row window over the whole project, newest change first. Filter by **kind**, **status**, and **review**, or jump to a preset: open accepted work, or the proposed review queue.

konspekt

local · read-only

Changes

Stats

Goals

PRESETS

Open + accepted work

Proposed review queue

Q

Filter by id...

KIND

goal

investigation

experiment

topic

task

note

concept

noteworthy

artifact

waypoint

STATUS

●

open

●

active

●

resolved

●

abandoned

REVIEW

proposed

accepted

Showing 10 of 152 entities · window of 10, scroll for the rest

●

 synced just now · 152 entities

The size of the project

152 entities, 257 edges, 75% accepted, 38 open proposals.

Accepted versus proposed by kind, and the oldest proposals still waiting on a human decision.

konspekt local · read-only

ChangesStatsGoals

152ENTITIES

257EDGES

75%ACCEPTED

38OPEN PROPOSALS

BY KIND — ACCEPTED VS PROPOSED

accepted

proposed

goal		7 · 5 acc · 2 prop
investigation		5 · 5 acc · 0 prop
task		36 · 25 acc · 11 prop
concept		14 · 13 acc · 1 prop
noteworthy		55 · 39 acc · 16 prop
artifact		21 · 15 acc · 6 prop
waypoint		14 · 12 acc · 2 prop

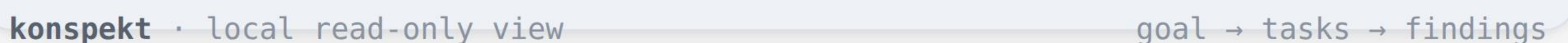
OLDEST UNACCEPTED PROPOSALS

nw-components-are-not-the-standard	noteworthy	61d
nw-node-status-does-transition	noteworthy	61d
nw-payload-reference-only-admits-enums	noteworthy	61d
artifact-schema-reconciliation	artifact	61d
artifact-components	artifact	60d
artifact-conformance-checker	artifact	60d
wp-conformance-checker	waypoint	60d
goal-collaboration	goal	22d
task-agent-fleet	task	22d
task-enterprise-persistence	task	22d

konspekt · local read-only view

counts over the graph

Each goal opens into its **decomposes** subgraph, drawn as a layered diagram. A node's border is its status; a dashed border means it is still only proposed.



Open any atom, read its record

Click any row or graph node for the whole atom: its **type, status, and review state**, the provenance pointer, and the body the agent wrote. Read-only, like everything here.

konspekt

local · read-only

PRESETS

Open + accepted work

Proposed review queue

KIND

goal

investigation

experiment

topic

task

note

concept

note

STATUS

open

active

resolved

abandoned

REVIEW

proposed

accepted

ENTITY ID	KIND	STATUS
artifact-implementation-zero-app	artifact	—
artifact-implementation-zero-design	artifact	—
nw-electron-shell	noteworthy	—
nw-implementations-directory	noteworthy	—
nw-implementation-layering	noteworthy	—
task-implementation-zero	task	active
task-intellij-plugin	task	open
artifact-ui-design	artifact	—
nw-poll-is-the-floor-push-is-optional	noteworthy	—
task-atom-vocabulary	task	open

Showing 10 of 152 entities · window of 10, scroll for the rest

STATUS

open

active

resolved

abandoned

REVIEW

proposed

awaiting a human disposition

accepted

human-accepted

GRAPH

node border = status · dashed border = p

edges are decomposes , goal at top

Read-only over the local working tree, refreshed by a filesystem watcher over Server-Sent Events. Changes is a window over every entity; Stats is a projection of counts and unaccepted-proposal age; Goals walks the decomposes subgraph. Click any entity for its file; nothing here writes to the instance.

task-implementation-zero

``yaml

id: task-implementation-zero

type: task

title: Build implementation_zero – a local watcher and floating read-only view

status: active

summary:

origin: machine

pinned: false

updatedAt: 2026-09-19T12:00:00Z

review: accepted

provenance:

sourceRef: db06867ee38876760cf3b6f2ab565ebb688a3cfd

contentHash:

db06867ee38876760cf3b6f2ab565ebb688a3cfd

conversationId: app-design-101

timestamp: 2026-09-19T12:00:00Z

confidence: 0.7

createdAt: 2026-09-19T12:00:00Z

updatedAt: 2026-09-19T12:00:00Z

``

Task: Build implementation_zero – a local watcher and floating read-only view

The first real build of the [[task-konspekt-ui-app]] view: a standalone process that runs beside the IDE, watches ``.konspekt/instance/`` on the local filesystem, and renders the one HTML view in an always-on-top window. Read-only – it reflects the working tree as the agent and human write to it and takes no disposition, so ``propose-accept`` stays intact because nothing writes.

Structure in two parts:

- **Core** (zero-dependency Node). A filesystem watcher over the working tree – the push variant [[nw-poll-is-the-floor-push-is-optional]] names – that debounces a burst of writes (a ``sync`` touches many files; a rename arrives as `create+delete` per [[nw-rename-fires-as-creation]]) and re-runs ``loadInstance()`` from ``lib/conformance.mjs``, the one shared reader, so there is no second parser. It serves the view and the entities over localhost and pushes a change signal over Server-Sent Events; the payload carries the fact of change and the page re-reads. This core is shell-agnostic and reused by later shells ([[nw-one-view-two-transport]]).

- **Always-on-top shell**. A native window wrapping the same localhost view, kept above other windows, chosen as part of this layer for adoption. Packaged with Electron ([[nw-electron-shell]]); the core above does not depend on it.

Scope boundary: local ``fs.watch`` sees local edits, not remote commits ([[nw-poll-is-the-floor-push-is-optional]]); remote fetch is out of scope. Writes and the authority verbs are [[task-task-workthrough-ui]], a later layer ([[nw-implementations-laverinall]]. The ``ui://`` card is

konspekt · local read-only view

one atom, in full

Who said what, behind a decision

Open any atom for the **verbatim exchange** it came from: the human and the assistant working the decision, turn by turn. Both sides, never paraphrased, each hop checked by re-hashing the source against its recorded hash.

konspekt

local · read-only

PRESETS

Open + accepted work

Proposed review queue

KIND

goal

investigation

experiment

topic

task

note

concept

note

STATUS

open

active

resolved

abandoned

REVIEW

proposed

accepted

ENTITY ID	KIND	STATUS
artifact-implementation-zero-app	artifact	—
artifact-implementation-zero-design	artifact	—
nw-electron-shell	noteworthy	—
nw-implementations-directory	noteworthy	—
nw-implementation-layering	noteworthy	—
task-implementation-zero	task	active
task-intellij-plugin	task	open
artifact-ui-design	artifact	—
nw-poll-is-the-floor-push-is-optional	noteworthy	—
task-atom-vocabulary	task	open

Showing 10 of 152 entities · window of 10, scroll for the rest

STATUS

open

active

resolved

abandoned

REVIEW

proposed

awaiting a human disposition

accepted

human-accepted

GRAPH

node border = status · dashed border = p

edges are decomposes , goal at top

Read-only over the local working tree, refreshed by a filesystem watcher over Server-Sent Events. Changes is a window over every entity; Stats is a projection of counts and unaccepted-proposal age; Goals walks the decomposes subgraph. Click any entity for its file; nothing here writes to the instance.

nw-provenance-content-addressed

conversations, attach actual chat text (my prompts and your responses) as provenance mechanism? I think all types of nodes would benefit from this

Human – options + git
the choices are 1. assume the vendors will expose deep links to the relevant parts of conversations or 2. they will serve up hashes that can be later use to find the parts or 3. bring the parts in "ourselves". We *could* cheat a little and use git and its commit hash mechanism to store the parts; (you'll be a good vendor and extract the parts for now). Specification-wise, we'll just say "a node has to have a pointer to an addressable text and the hashes on that text have to match the hash stored at the construction time"

Assistant – git is content-addressed; address and hash are two fields
"git *is** a content-addressed object store, so if we bring the parts in as blobs, the blob SHA already *is** the content hash. We don't define a hashing function, we don't maintain a `Source` entity with its own digest machinery, and 'the hash matches' stops being a check we run – it's structural."

"Address and hash want to be two fields, even though git tempts you to collapse them into one." Deep link: the text "can drift ... So you *must** store an independent construction-time hash to detect drift. Address ≠ hash." Git: "the SHA *is** the content hash. Collapses cleanly."

Verify probe: "an entity's provenance verifies iff hashing the text resolved at its `sourceRef` reproduces its stored `contentHash`."

Human – ordering + fields
ordering will be solved by timestamps. the "parts" is not very deterministic, but then again - nothing in LLM space is and, the worst case, we'll add UI manual curation support. We do want source ref and content hash, as long as source ref points me to the files you put in git

Assistant – push-based inverts idempotence; the source is the stable layer
"Bringing parts in is *push-based** ingestion: at extraction time I write the source excerpt into git alongside the atoms. That means I never re-read a durable already-consumed source, so the skip-watermark – the thing that needed an orderable key – stops being load-bearing for correctness. What carries correctness instead is match-level dedup on the source blob..."

"the *source excerpt itself** is near-deterministic, because copying text is close to deterministic even when interpretation

ONE LOCAL WINDOW

Read-only, local, live over the working tree

Changes	a rolling to-do over every entity, filtered by kind, status, review
Stats	size at a glance and the oldest proposals still waiting
Goals	each goal decomposed into its subgraph
Detail	the record and the verbatim source it came from, hash-verified

NEXT RELEASE

an **IntelliJ plugin for the engineer persona** — the same view in the IDE, where writes arrive first.

github.com/finos-labs/konspekt

A filesystem watcher feeds it over server-sent events. It takes no disposition, so propose → accept still holds.

#AI · #KnowledgeGraphs · #OpenSource · #FINOS · #konspekt